

1 Từ chuỗi tới attention (From Sequences to Attention)

PHỤ THUỘC C/12/02-lam-cho-training-chay-duoc (tích các Jacobian — vanishing gradient ở đây là đúng hiện tượng đó, chỉ trải theo thời gian); C/13/01-dong-cnn (khái niệm trường tiếp nhận và inductive bias).

TÓM TẮT

Mục này đi từ mạng hồi quy tới **attention** và tới các mô hình không gian trạng thái, đọc như một chuỗi gỡ nút thắt chứ không như một danh sách kiến trúc. Đọc xong bạn giải thích được vì sao **transformer** bỏ được tính tuần tự, và vì sao nó phải trả bằng chi phí bậc hai. Bạn cũng nói được ba hướng tấn công chi phí đó khác nhau ở tầng nào: toán, kiến trúc, hay hệ thống. Nếu chỉ nhớ một điều: LSTM và ResNet giải cùng một bài toán bằng cùng một mẹo. Cả hai tạo một *đường cộng* để gradient đi qua mà không bị nhân dồn. Nhận ra điều đó thì hai kiến trúc trông rất khác nhau trở thành một ý tưởng duy nhất.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi (sequence)** — dữ liệu có thứ tự, ví dụ một câu gồm nhiều từ hoặc một video gồm nhiều khung hình; “thứ tự là thông tin” nghĩa là hoán vị các phần tử sẽ đổi ý nghĩa.
- **token** — một phần tử của chuỗi sau khi đã được cắt và mã hoá thành vectơ số — một từ, một mảnh từ, hoặc với ảnh là một ô vuông 16×16 pixel.
- **embedding** — vectơ số biểu diễn một token; các token có nghĩa gần nhau được đặt gần nhau trong không gian này, và toàn bộ mô hình chỉ làm việc với vectơ chứ không với ký tự.
- **trạng thái ẩn (hidden state)** — vectơ mà mạng hồi quy mang theo từ bước thời gian này sang bước sau; nó là toàn bộ “trí nhớ” của mạng, nên mọi thứ không nhét được vào nó là mất.
- **BPTT (backpropagation through time)** — cách áp lan truyền ngược cho mạng hồi quy: trải mạng ra thành T bản sao theo thời gian rồi lan truyền ngược qua cả T bước, nên một chuỗi dài 100 hành xử như một mạng sâu 100 tầng.
- **attention** — cơ chế cho mỗi vị trí tự chọn lấy thông tin từ mọi vị trí khác bằng một trọng số học được, thay vì phải chờ thông tin truyền dần qua từng bước.
- **độ phức tạp bậc hai** — chi phí tỉ lệ với T^2 khi độ dài chuỗi là T ; gấp đôi độ dài thì chi phí gấp bốn, nên đây là rào cản chính khi mang attention sang ảnh độ phân giải cao.
- **HBM** — bộ nhớ băng thông cao gắn ngoài chip GPU — rất lớn nhưng chậm hơn nhiều so với bộ nhớ trên chip; số lần đọc ghi HBM thường quyết định tốc độ hơn cả số phép nhân.

A. Đặt vấn đề

Convolution giả định dữ liệu nằm trên một lưới đều và quan hệ quan trọng là quan hệ gần. Nhưng nhiều dữ liệu có *thứ tự* chứ không có lưới, và quan hệ quan trọng có thể ở rất xa: từ đầu câu quyết định nghĩa từ cuối câu, khung hình đầu video quyết định cách hiểu khung hình cuối. Câu hỏi của mục này là: cấu trúc nào xử lý được loại quan hệ đó, và mỗi câu trả lời phải trả giá bằng gì?

Giá trị cụ thể của mục: nhìn một kiến trúc xử lý chuỗi, bạn trả lời được ba câu. Thông tin đi từ vị trí i tới vị trí j qua bao nhiêu bước? Chi phí tăng theo độ dài như thế nào? Và cái gì trong kiến trúc đang giữ vai trò “biết thứ tự”? Ba câu đó đủ để phân loại mọi biến thể mà bạn sẽ gặp.

TRỰC GIÁC

Hãy đo mọi kiến trúc trong mục này bằng một đại lượng duy nhất: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Trong mạng hồi quy, thông tin phải đi qua T trạm nối tiếp, và

mỗi trạm nhân nó với một ma trận. Giống như truyền một tin nhắn qua dây chuyền thì thăm dài 100 người. Trong attention, mọi vị trí nói chuyện trực tiếp với mọi vị trí, đường đi bằng 1 — giống như thay dây chuyền thì thăm bằng một phòng họp mà ai cũng nghe thấy ai. Toàn bộ ưu điểm lẫn toàn bộ chi phí của transformer nằm gọn trong phép đổi đó: phòng họp không tam sao thất bản, nhưng số cặp người phải nghe nhau tăng theo bình phương số.

B. Mạch từ RNN tới state space model

Sáu bước dưới đây là một chuỗi liên tục, và mỗi bước chỉ hiểu được khi đã thấy vấn đề mà bước trước để lại.

I Mạch 2 — từ chuỗi tới attention

- 1 Vấn đề** dữ liệu có thứ tự, nhưng mạng dày đặc và convolution đều không có khái niệm “bước trước”.

Giải pháp **mạng hồi quy** (*recurrent neural network, RNN*): giữ một trạng thái ẩn $h_t = f(W h_{t-1} + U x_t)$ và cập nhật nó theo từng bước, nên cùng một bộ trọng số dùng lại cho mọi độ dài chuỗi.

Vấn đề mới huấn luyện bằng BPTT nghĩa là lan truyền ngược qua T tầng dùng chung trọng số. Gradient vì thế tỉ lệ với lũy thừa T của cùng một ma trận. Đó đúng là hiện tượng vanishing và exploding của mục C/12/02, nhưng tệ hơn: thừa số lặp lại y hệt nhau nên không có triệt tiêu ngẫu nhiên nào cứu.
- 2 Vấn đề** trí nhớ dài không tồn tại: sau vài chục bước tín hiệu đã tắt.

Giải pháp **LSTM** [1] và GRU dùng **cổng** (*gate*) để tạo một trạng thái ô $c_t = f_t \odot c_{t-1} + i_t \odot g_t$. đạo hàm $\partial c_t / \partial c_{t-1} = f_t$, nên khi cổng quên mở gần 1 thì gradient chảy qua mà *không* bị nhân với ma trận trọng số. Đây chính xác là mẹo của kết nối tắt trong ResNet, chỉ khác là hệ số của đường cộng ở đây học được thay vì cố định bằng 1.

Vấn đề mới tính tuần tự vẫn còn nguyên: h_t cần h_{t-1} , nên không song song hoá được theo thời gian, và độ dài dùng được vẫn giới hạn.
- 3 Vấn đề** tuần tự nghĩa là không tận dụng được GPU, và thông tin xa vẫn phải đi qua mọi trạm trung gian.

Giải pháp **attention**. Mỗi vị trí sinh một truy vấn q ; mọi vị trí sinh khoá k và giá trị v . Đầu ra là tổ hợp $\text{softmax}(q K^\top / \sqrt{d}) V$, tức truy cập trực tiếp mọi vị trí trong một bước. **Transformer** [2] bỏ hẳn phần hồi quy và chỉ giữ attention cộng mạng truyền thẳng theo vị trí, nên toàn bộ chuỗi được xử lý song song.

Giá phải trả chi phí và bộ nhớ bậc hai theo độ dài, và mất hoàn toàn khái niệm thứ tự vì phép tính bất biến với hoán vị.
- 4 Vấn đề** mô hình không còn biết token nào đứng trước token nào.

Giải pháp **mã hoá vị trí** (*positional encoding*): cộng hoặc trộn thêm thông tin vị trí vào embedding.

Giả định vị trí có thể mã hoá thành một vectơ mà mô hình học cách dùng.

Hệ quả bốn họ với đánh đổi khác nhau — tuyệt đối dạng sin, tuyệt đối học được, tương đối, và RoPE quay embedding theo góc tỉ lệ vị trí [3]. Tuyệt đối đơn giản nhưng ngoại suy kém ra ngoài độ dài đã thấy khi huấn luyện; tương đối và RoPE ngoại

suy tốt hơn vì chúng mã hoá *khoảng cách* chứ không mã hoá chỉ số. Với ảnh còn cần biến thể hai chiều, vì vị trí là một cặp (x, y) chứ không phải một số.

5 Vấn đề

Giải pháp

Hệ quả

chi phí bậc hai chặn đường tới ảnh độ phân giải cao.

ba hướng tấn công ở ba tầng khác nhau — xấp xỉ toán học, giới hạn phạm vi kiến trúc, và tối ưu truy cập bộ nhớ — xem Bảng 1.

đây là một bài tập rất tốt về việc *cùng một vấn đề có thể tấn công ở nhiều tầng trừu tượng*, và ba lời giải không loại trừ nhau: người ta dùng FlashAttention *bên trong* Swin.

6 Vấn đề

Giải pháp

Giá phải trả

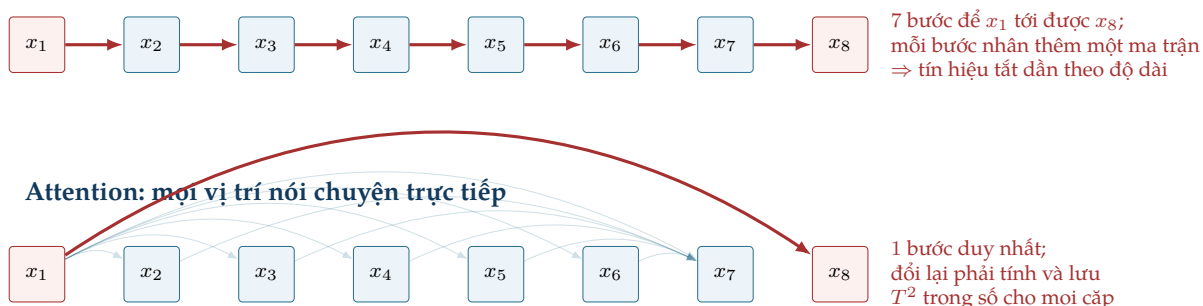
cả ba hướng trên đều vá chứ không xoá được bậc hai.

mô hình không gian trạng thái (*state space model, SSM*) như Mamba [4] quay lại dạng hồi quy tuyến tính, với tham số phụ thuộc đầu vào. Nó tính được bằng phép quét song song. Nhờ vậy chi phí tuyến tính theo độ dài mà vẫn huấn luyện song song được.

đây là hướng đang lên, chưa phải kết luận: tính tới thời điểm viết, SSM chưa thay thế attention trong thị giác, và phần lớn kết quả mạnh nhất vẫn thuộc về các kiến trúc dựa trên attention.

↪ cùng nguyên lý với C/13/01 — cổng của LSTM và kết nối tắt của ResNet là cùng một mẹo — tạo một đường cộng để gradient không bị nhân dồn

Mạng hồi quy: thông tin bò từng bước



Hình 1. Đại lượng đáng dùng để so mọi kiến trúc chuỗi: *độ dài đường đi của thông tin* giữa hai vị trí xa nhau. Ở mạng hồi quy nó bằng khoảng cách giữa hai vị trí, nên tín hiệu phải sống sót qua từng ấy phép nhân ma trận — đây chính là vanishing gradient của mục C/12/02, trái theo trục thời gian. Ở attention nó luôn bằng 1, và cái giá là các cạnh mờ trong hình: mọi cặp đều phải có một trọng số, nên số cạnh tăng theo T^2 . LSTM nằm giữa hai thái cực: đường đi vẫn dài, nhưng cổng quên tạo một đường cộng nên tín hiệu không bị nhân dồn.

Hình 1 là dạng hình học của ẩn dụ ở đầu mục, và nó cũng cho thấy vì sao ba bước đầu của mạch phải đi theo đúng thứ tự đó.

C. Vì sao chia cho $\sqrt{d}$ — một phép tính hai dòng

Chi tiết $\sqrt{d}$ trong công thức attention trông như một hệ số làm đẹp. Nhưng bỏ nó đi thì mô hình không huấn luyện được, và lý do tính ra được. Giả sử các thành phần của q và k độc lập, trung bình 0, phương sai 1. Tích vô hướng $q \cdot k = \sum_{i=1}^d q_i k_i$ là tổng của d số hạng độc lập, mỗi số hạng trung bình 0 và phương sai 1. Nên nó có phương sai d , tức độ lệch chuẩn $\sqrt{d}$. Với $d = 64$, độ lệch chuẩn của các logit là 8.

Ý nghĩa của con số đó: softmax trên một tập logit có độ lệch chuẩn 8 gần như luôn dồn toàn bộ khối lượng vào một phần tử, tức nó bão hoà thành một vectơ gần one-hot. Mà đạo hàm của softmax ở vùng bão hoà xấp xỉ 0 — đúng hiện tượng đã gặp với sigmoid ở $C/12/02$. Chia cho $\sqrt{d}$ đưa độ lệch chuẩn về 1, giữ softmax ở vùng có gradient. Nói gọn: $\sqrt{d}$ không phải chuẩn hoá cho đẹp, nó là điều kiện để attention có gradient khi d lớn.

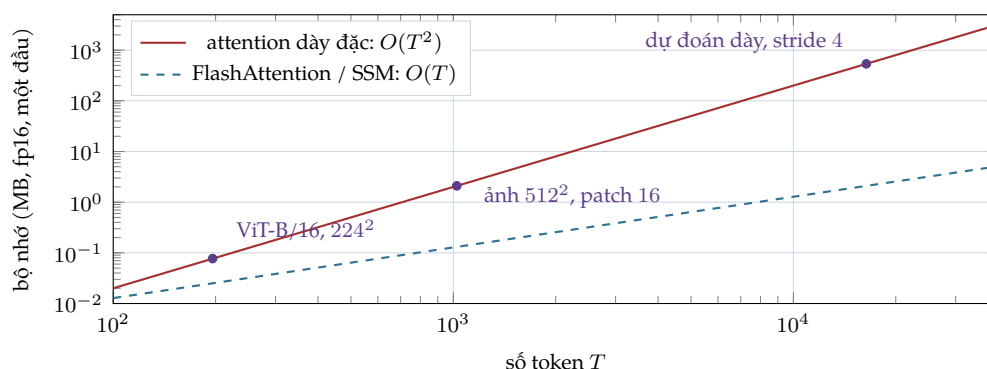
↔ trường hợp riêng của $C/12/02$ — đây đúng là hiện tượng bão hoà của sigmoid, chỉ khác là xảy ra ở softmax và nguyên nhân là thang của tích vô hướng

D. Chi phí bậc hai: con số cụ thể trước, giải pháp sau

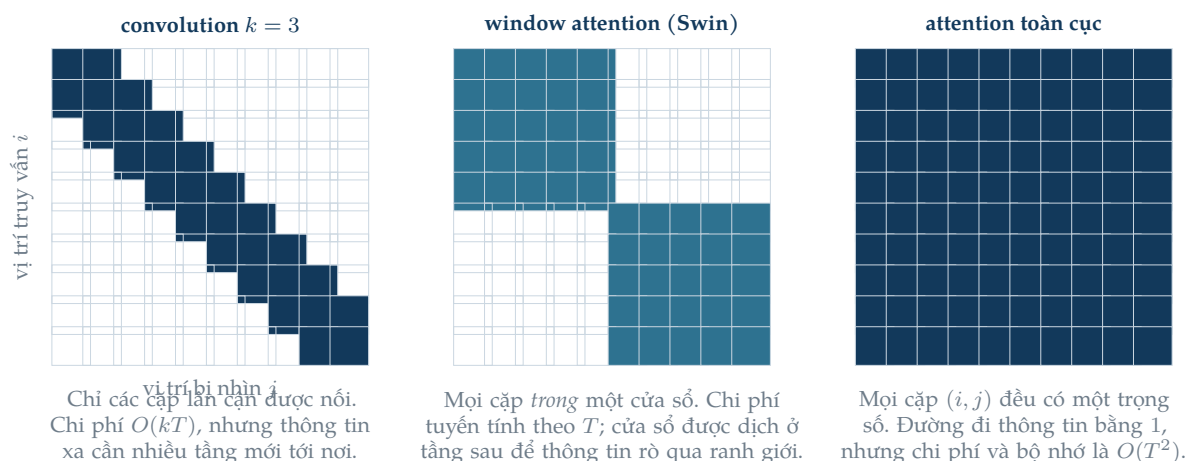
Trước khi bàn giải pháp, hãy làm phép tính để biết vấn đề nghiêm trọng tới mức nào. Ma trận attention có kích thước $T \times T$ cho mỗi đầu và mỗi tầng.

- **Phân loại ảnh, ViT-B/16 ở 224×224 :** $T = (224/16)^2 = 196$ token, nên ma trận có $196^2 = 38\,416$ phần tử — không đáng kể.
- **Ảnh 512×512 , patch 16:** $T = 1024$, ma trận có hơn một triệu phần tử. Vẫn chạy được.
- **Dự đoán dài ở stride 4 trên ảnh 512×512 :** $T = 128^2 = 16\,384$, ma trận có $2,68 \cdot 10^8$ phần tử, tức khoảng 537 MB ở fp16 — cho một đầu, một tầng. Nhân với 12 đầu là 6,4 GB chỉ để chứa các trọng số attention của một tầng.

Con số cuối là toàn bộ lý do các biến thể ở Bảng 1 tồn tại, và cũng là lý do ViT phải không dùng trực tiếp được cho segmentation. Hình 2 vẽ cùng ba mốc đó trên trục log để thấy chỗ đường cong bẻ gãy ngân sách.



Hình 2. Bộ nhớ cần để chứa ma trận attention theo số token, thang log hai chiều. Ba chấm là ba chế độ dùng thật. Khoảng cách giữa hai đường là lý do tồn tại của cả ba hướng tấn công ở Bảng 1; lưu ý FlashAttention nằm trên đường tuyến tính về bộ nhớ nhưng vẫn làm đủ $O(T^2)$ phép tính.



Hình 3. Ba kiến trúc, cùng một cách nhìn: ô (i, j) được tô nghĩa là vị trí i được phép lấy thông tin trực tiếp từ vị trí j . Convolution là một dải hẹp quanh đường chéo — đó chính là “tính cục bộ” phát biểu dưới dạng ma trận. Window attention là các khối trên đường chéo, tức một thoả hiệp: dày đặc trong cửa sổ, rỗng ngoài cửa sổ. Attention toàn cục tô kín, và diện tích phần tô chính là chi phí phải trả. Nhìn ba lưới cạnh nhau thì “inductive bias” thôi không còn là một khái niệm trừu tượng: nó chính là các ô bị bỏ trống vì kiến trúc quyết định trước rằng chúng không đáng quan tâm.

Hình 3 nên được xem trước khi đọc Bảng 1: hai trong ba hướng tấn công ở đó chính là hai cách khác nhau để *bớt tô* lưới bên phải, còn hướng thứ ba giữ nguyên lưới nhưng đổi cách nó được tính.

Bảng 1. Ba hướng tấn công chi phí bậc hai, ở ba tầng trừu tượng khác nhau. Chúng không loại trừ nhau và trên thực tế hay được dùng chồng lên nhau.

Tầng tấn công	Đại diện	Cơ chế	Mất gì / hỏng khi nào
Toán học	Linear attention	Đổi thứ tự nhân ma trận nhờ một xấp xỉ của softmax, biến $O(T^2d)$ thành $O(Td^2)$	Chất lượng giảm ở tác vụ cần attention “sắc”, tức cần chọn đúng một vị trí trong nhiều vị trí giống nhau
Kiến trúc	Swin, window attention [5]	Chỉ cho attention trong cửa sổ cố định, rồi dịch cửa sổ giữa các tầng để thông tin rò qua	Quan hệ xa cần nhiều tầng mới nối được; xuất hiện lại một dạng “đường đi dài” mà attention sinh ra để xoá
Hệ thống	FlashAttention [6]	Chia ô và tính softmax trực tuyến để không bao giờ dựng ma trận $T \times T$ trong HBM	Không giảm FLOPs và không giảm độ phức tạp tiệm cận; lợi ích phụ thuộc kích thước SRAM của GPU cụ thể

Bảng 1 đáng đọc kỹ vì nó dạy một thói quen chung hơn cả nội dung: khi gặp một rào cản, hãy hỏi nó có thể bị tấn công ở tầng nào. FlashAttention là ví dụ sạch nhất — nó không sửa thuật toán, nó sửa *cách thuật toán chạm vào bộ nhớ*, và với người đã tối ưu kernel CUDA thì đây là câu chuyện quen thuộc: tăng cường độ số học bằng cách giữ dữ liệu trên chip và giảm số vòng đọc ghi ra bộ nhớ ngoài.

↔ **ràng buộc bởi** B/11 — FlashAttention chỉ có ý nghĩa trong mô hình roofline: nó dời kernel từ phía ngiên bằng thông sang phía ngiên tính toán

E. Giới hạn và trường hợp hỏng

Attention không phải luôn thắng, và ba chỗ nó thua đáng nhớ.

Thứ nhất, với chuỗi ngắn và dữ liệu ít, mạng hồi quy vẫn cạnh tranh. Chi phí bậc hai chỉ là vấn đề khi T lớn; khi T nhỏ, phần chi phí thật nằm ở mạng truyền thẳng chứ không ở attention, còn transformer thì lại cần nhiều dữ liệu hơn vì nó cài ít giả định hơn.

Thứ hai, mã hoá vị trí là chỗ rò rỉ giả định. Mọi phép ngoại suy ra độ dài chưa từng thấy khi huấn luyện đều là ngoại suy, và mô hình có thể xuống cấp lạng lẽ. Với ảnh, đổi độ phân giải đầu vào mà không nội suy lại positional embedding là một lỗi phổ biến, cho ra kết quả tệ mà không báo lỗi.

Thứ ba, linear attention và window attention đánh đổi ở hai chỗ khác nhau, nên đừng coi chúng thay thế được nhau. Linear attention mất khả năng chọn sắc; window attention giữ được độ sắc trong cửa sổ nhưng mất quan hệ xa. Nếu tác vụ của bạn cần cả hai — ví dụ theo vết một đối tượng nhỏ qua toàn ảnh — thì cả hai đều không đủ và bạn phải quay lại kiến trúc phân cấp.

Về mức độ tin cậy: các con số bộ nhớ ở mục D là số học thuần túy nên chắc chắn. Còn phát biểu “SSM chưa thay thế attention trong thị giác” chỉ là đánh giá của tôi tính tới tháng 9/2026. Nó dựa trên việc các kết quả dần dần vẫn dùng attention, và có thể sai trong vòng một năm.

CẠM BẤY

Cạm bẫy thực hành hay gặp nhất khi mang một ViT sang bài toán của mình: đổi kích thước ảnh đầu vào mà quên nội suy positional embedding. Số token thay đổi, nhưng bảng vị trí đã học vẫn có kích thước cũ; tùy cài đặt mà bạn nhận một exception — trường hợp may mắn — hoặc một phép cắt/dệm âm thầm cho ra một mô hình hoạt động nhưng kém hẳn. Dấu hiệu nhận biết: điểm số tụt đúng một bậc ngay khi đổi độ phân giải, trong khi mọi thứ khác giữ nguyên.

CẦN NHỚ

Đo mọi kiến trúc chuỗi bằng ba câu: thông tin đi từ i tới j qua bao nhiêu bước, chi phí tăng theo T như thế nào, và cái gì đang giữ vai trò “biết thứ tự”. RNN cho đường đi $O(T)$ và chi phí $O(T)$; attention cho đường đi $O(1)$ và chi phí $O(T^2)$; mọi biến thể là một điểm nằm giữa hai thái cực đó. [T2]

F. Kết nối

Bước hai của mạch — cổng tạo đường cộng — là cùng một ý tưởng với kết nối tắt ở C/13/01-dong-cnn, và cả hai đều là hệ quả của lập luận tích Jacobian ở C/12/02-lam-cho-training-chay-duoc; nếu chỉ nhớ một liên kết trong cả chương 13 thì nên nhớ liên kết này. Mục tiếp theo, C/13/03-vit-va-inductive-bias, mang attention vào ảnh và hỏi cái giá của việc vứt bỏ tính cục bộ. Phần FlashAttention nối trực tiếp sang B/11-gioi-han-tinh-toan (cường độ số học, roofline) và sang D/20-toi-uu-va-nen-mo-hinh. Còn attention với vai trò cơ chế lựa chọn trong bài toán thị giác cụ thể xuất hiện lại ở C/14-bai-toan-cv-loi qua DETR [7].

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao vanishing gradient trong RNN tệ hơn trong một mạng truyền thẳng cùng độ sâu, dù cơ chế là như nhau?
2. Cổng của LSTM và kết nối tắt của ResNet giống nhau ở điểm cốt lõi nào, và khác nhau ở điểm nào?
3. Nếu bỏ hệ số $1/\sqrt{d}$ trong attention thì hỏng ở đâu, và vì sao hậu quả nặng hơn khi d lớn?

4. FlashAttention không giảm FLOPs — vậy nó nhanh hơn nhờ đâu, và trên phần cứng nào lợi ích của nó nhỏ đi?
5. Ba hướng tấn công chi phí bậc hai nằm ở ba tầng khác nhau; với bài toán segmentation ảnh 1024×1024 thì bạn chọn hướng nào trước, và vì sao?
6. Bạn fine-tune một ViT ở độ phân giải gấp đôi lúc pretrain và kết quả tệ đi. Nghi phạm số một là gì?

Nguồn

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [3] J. Su, M. Ahmed, Y. Lu, et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, 2024.
- [4] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [5] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [6] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [7] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.